
Documentação de Projeto

para o sistema

Stream Sentry

Versão 1.2

Projeto de sistema elaborado pelo aluno Rafael Parreira Chequer e apresentado ao curso de Engenharia de Software da PUC Minas como parte do Trabalho de Conclusão de Curso (TCC) sob orientação dos professores Cleiton Silva Tavares, Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso e Raphael Ramos Dias Costa.

02 de novembro de 2025

Tabela de Conteúdo

Tabela de Conteúdo

Histórico de Revisões

1. Introdução

2. Modelo de Usuários e Requisitos

- 2.1 Descrição de Atores
- 2.2 Modelos de Usuário
- 2.3 Modelo de Casos de Uso e Histórias de Usuários
- 2.4 Diagrama de Sequência do Sistema e Contrato de Operações

3.

Diagramas

- 3.1 Diagrama de Classes
- 3.2 Diagramas de Sequência
- 3.3 Diagramas de Comunicação
- 3.4 Arquitetura Lógica
- 3.5 Diagrama de Atividade
- 3.6 Diagrama de Componentes e Implantação

4. Projeto de Interface com Usuário

- 4.1 Interfaces Comuns a Todos os Atores

5. Modelos de Projeto

- 5.1 Modelo de Dados
- 5.2 Glossário

6. Cronograma

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	17/09/2025	Criação inicial do documento para entrega da A3, contendo diagrama de casos de uso, modelos de usuários e projeto de tela.	1.0
Rafael Parreira Chequer	13/10/2025	Criação do diagrama de sequência de sistema, diagrama de classes, diagrama de sequência e diagrama de comunicação e adição das demais interfaces.	1.1
Rafael Parreira Chequer	02/11/2025	Adição da arquitetura lógica, diagrama de atividades, componentes e implantação. Inclusão do modelo de dados e do glossário.	1.2

1. Introdução

Este documento apresenta a análise de usuário, contexto, interface e interação do sistema Stream Sentry, uma ferramenta web *open-source* (licença MIT) desenvolvida em React para automação de testes *end-to-end* em aplicações de videoconferência baseadas em *WebRTC* ou *APIs* de videoconferência (ex.: Zoom SDK, Jitsi). A referência principal para a descrição do problema, domínio e requisitos é o Documento de Visão (anexo a este documento), que detalha o escopo, as funcionalidades, os usuários-alvo (desenvolvedores React, engenheiros de QA (*Quality Assurance*), equipes ágeis e pesquisadores acadêmicos) e as restrições do sistema.

2. Modelos de Usuário e Requisitos

Esta seção apresenta os modelos de usuário e os requisitos funcionais do sistema Stream Sentry, com base nas necessidades identificadas no Documento de Visão. São descritos os atores envolvidos, personas representativas, casos de uso, histórias de usuário, diagramas de interação e contratos de operações, garantindo alinhamento entre as expectativas dos usuários e as funcionalidades do sistema. O objetivo é fornecer uma base clara e estruturada para o desenvolvimento, priorizando usabilidade, automação e extensibilidade em cenários de teste de videoconferência.

2.1 Descrição de Atores

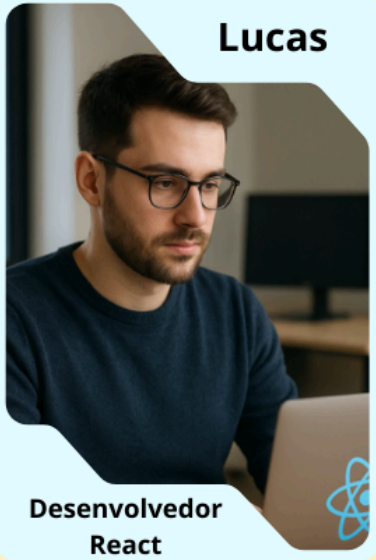
Os atores que interagem com o sistema Stream Sentry são definidos com base nos usuários-alvo descritos no Documento de Visão. Abaixo está a descrição de cada ator:

- **Desenvolvedor React:** Profissional que desenvolve aplicações de videoconferência em React, utilizando *WebRTC* ou *APIs* como Zoom SDK ou Jitsi. Este ator utiliza o Stream Sentry para automatizar testes *end-to-end*, reduzindo o tempo e o esforço em validações manuais.
- **Engenheiro de QA:** Responsável por garantir a qualidade de aplicações de videoconferência, utilizando o sistema para executar testes automatizados e analisar relatórios detalhados com métricas como latência, falhas e cobertura de testes.
- **Pesquisador Acadêmico:** Estudante ou professor que investiga automação de testes multimídia, utilizando o **Stream Sentry** como uma ferramenta *open-source* para experimentação e análise.
- **Equipe Ágil:** Grupo de profissionais (desenvolvedores, testadores e gerentes) que utilizam o sistema em fluxos de trabalho ágeis para integrar testes automatizados e relatórios em seus processos.

2.2 Modelos de Usuários

Para representar os usuários do sistema, foram desenvolvidas personas que refletem as características, necessidades e objetivos dos atores identificados. As personas são baseadas nas necessidades levantadas no Documento de Visão e ajudam a guiar o design do sistema.

Persona 1:



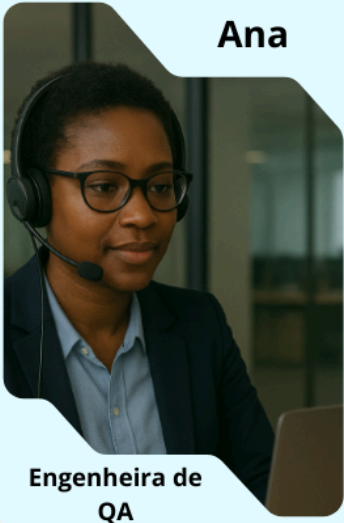
Lucas

**Desenvolvedor
React**

- **Idade:** 28 anos
- **Formação:** Bacharel em Ciência da Computação
- **Contexto:** Trabalha em startup desenvolvendo aplicações WebRTC
- **Objetivos:** Automatizar testes end-to-end para estabilidade e redução de bugs
- **Necessidades:** Interface simples, relatórios JSON/CSV, suporte a WebRTC
- **Frustrações:** Testes manuais demorados, ferramentas pagas caras
- **Cenário de Uso:** Configura teste com 5 usuários virtuais no StreamSentry e exporta relatório JSON

Figura 1: Lucas Desenvolvedor React

Persona 2:



Ana

Engenheira de QA

- **Idade:** 32 anos
- **Formação:** Engenharia de Software
- **Contexto:** Valida qualidade de áudio/vídeo em APIs como Jitsi
- **Objetivos:** Executar testes automatizados com métricas de latência e falhas
- **Necessidades:** Relatórios detalhados, interface intuitiva, suporte a headless browsers
- **Frustrações:** Ferramentas como Selenium não otimizadas para WebRTC
- **Cenário de Uso:** Configura teste de estabilidade com 10 usuários e exporta relatório CSV

Figura 2: Ana Engenheira de QA

Persona 3:



Carla

Pesquisadora Acadêmica

- **Idade:** 40 anos
- **Formação:** Doutora em Ciência da Computação
- **Contexto:** Pesquisa testes multimídia com ferramentas open-source
- **Objetivos:** Testar hipóteses com ferramenta flexível
- **Necessidades:** Código open-source (MIT), documentação, suporte a Zoom SDK
- **Frustrações:** Ferramentas proprietárias e falta de documentação
- **Cenário de Uso:** Configura teste com Jitsi, modifica código e documenta resultados

Figura 3: Carla Pesquisadora Acadêmica

Persona 4:



Felipe

Tech Lead

- **Idade:** 35 anos
- **Formação:** Engenharia de Software com MBA em Gestão Ágil
- **Contexto:** Tech Lead de startup de videoconferência, coordenando CI/CD
- **Objetivos:** Automatizar testes WebRTC, reduzir tempo de feedback de qualidade
- **Necessidades:** Integração com CircleCI, suporte Kubernetes, notificações Teams
- **Frustrações:** Testes falhando com codecs, configuração em containers complexa
- **Cenário de Uso:** Integra StreamSentry no pipeline e valida chamadas.

Figura 4: Lucas Tech Lead

2.3 Modelo de Casos de Uso e Histórias de Usuários

O diagrama de casos de uso do sistema Stream Sentry foi desenvolvido para representar as interações principais dos atores com o sistema, conforme as funcionalidades descritas no Documento de Visão. O diagrama está disponível na pasta Artefatos do repositório com o nome UseCaseDiagram.puml e é apresentado na Figura 1.

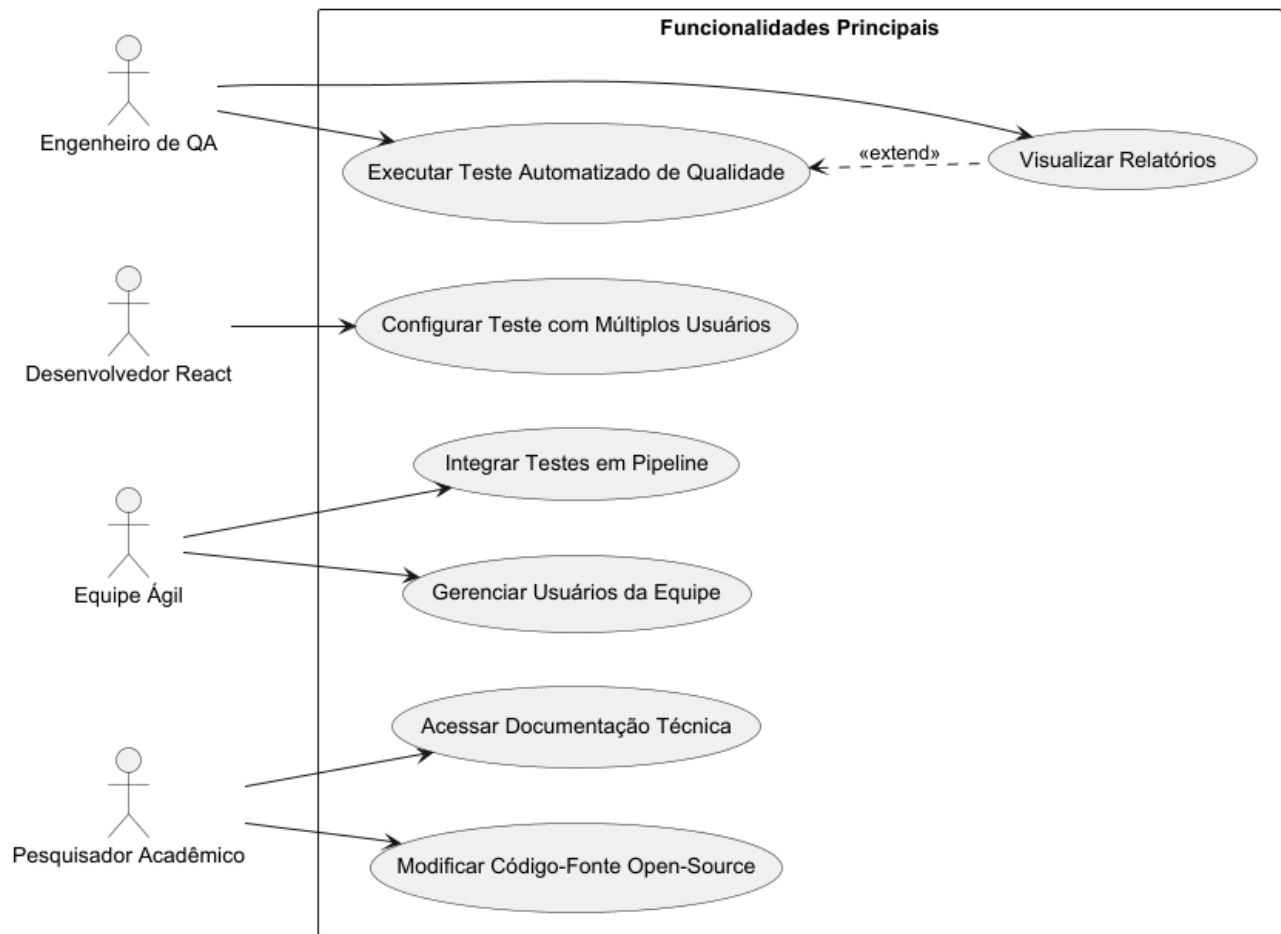


Figura 5: Diagrama de Casos de Uso do Stream Sentry

O diagrama ilustra os atores (Desenvolvedor React, Engenheiro de QA, Pesquisador Acadêmico, Equipe Ágil) e suas interações com os casos de uso principais: configurar testes, executar testes automatizados, visualizar relatórios e modificar código-fonte. Cada caso de uso reflete uma funcionalidade crítica ou importante identificada no Documento de Visão.

Abaixo está a lista de histórias de usuário, mapeadas com base nas necessidades e funcionalidades do sistema, utilizando identificadores únicos para referência:

- **US01:** Como Desenvolvedor React, eu quero configurar um teste com múltiplos usuários virtuais, para verificar a estabilidade da conexão em uma aplicação *WebRTC*, garantindo que a funcionalidade seja testada rapidamente.
- **US02:** Como Engenheiro de QA, eu quero executar testes automatizados de qualidade de áudio e vídeo, para identificar falhas de latência ou interrupções antes da entrega do software.

- **US03:** Como Engenheiro de QA, eu quero visualizar relatórios detalhados em formato JSON ou CSV, para compartilhar métricas de teste com a equipe e tomar decisões baseadas em dados.
- **US04:** Como Pesquisador Acadêmico, eu quero acessar o código-fonte do Stream Sentry, para modificá-lo e experimentar novos cenários de teste multimídia, atendendo aos meus objetivos de pesquisa.
- **US05:** Como membro de uma Equipe Ágil, eu quero configurar testes via uma interface web intuitiva, para integrar a automação ao fluxo de trabalho do time sem complicações.
- **US06:** Como Desenvolvedor React, eu quero que o sistema suporte *APIs* de videoconferência, para testar aplicações específicas sem necessidade de ferramentas adicionais.
- **US07:** Como Pesquisador Acadêmico, eu quero acessar documentação técnica completa, para entender a arquitetura do sistema e facilitar sua modificação para experimentos acadêmicos.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

O diagrama de sequência foi revisado para ser ultra compacto, com apenas dois participantes (Desenvolvedor React e Stream Sentry Core) e três interações, reduzindo significativamente a "horizontalidade". Ele foca no fluxo principal do caso de uso "Configurar e Executar Teste", com a integração com a *API* de videoconferência (ex.: Jitsi) implícita no Stream Sentry Core.

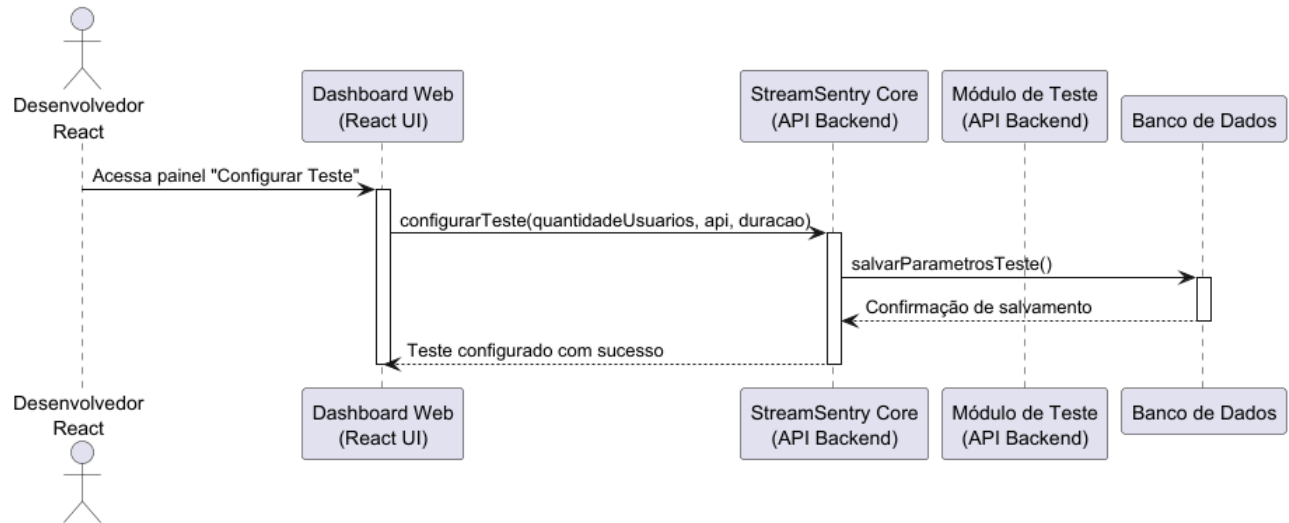


Figura 6: Diagrama de Sequência para Configuração de Testes

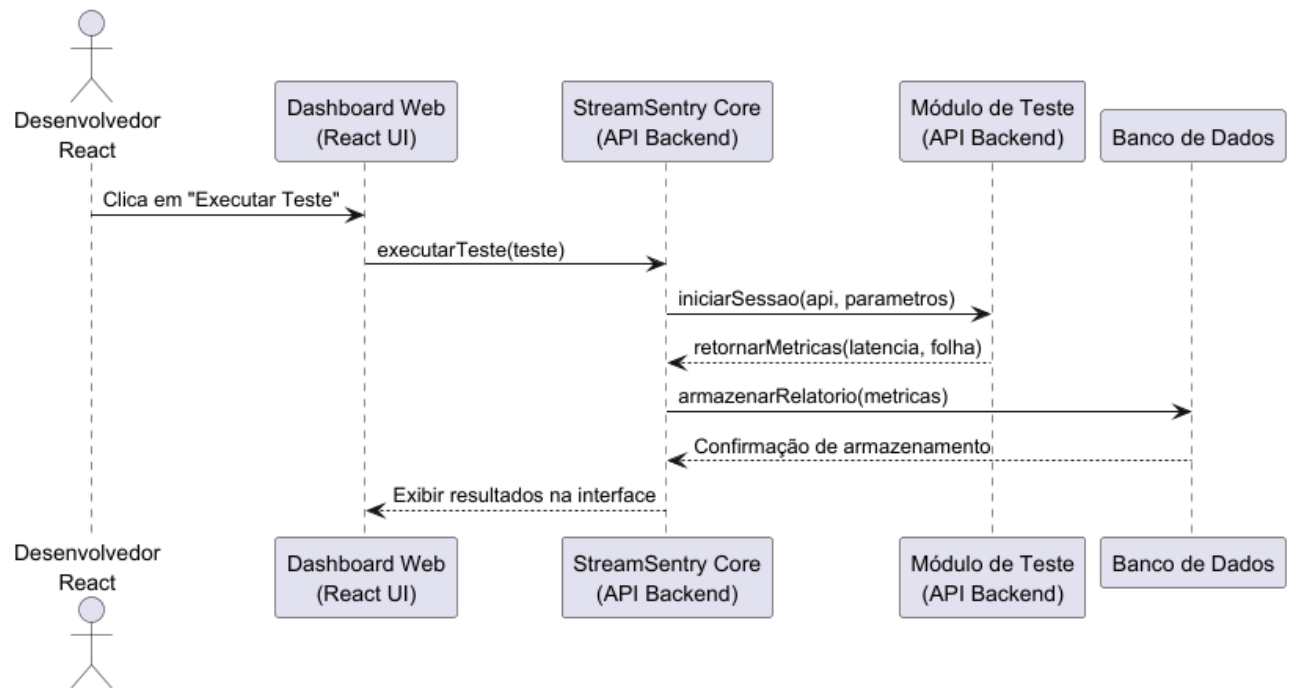


Figura 7: Diagrama de Sequência para Execução de Testes

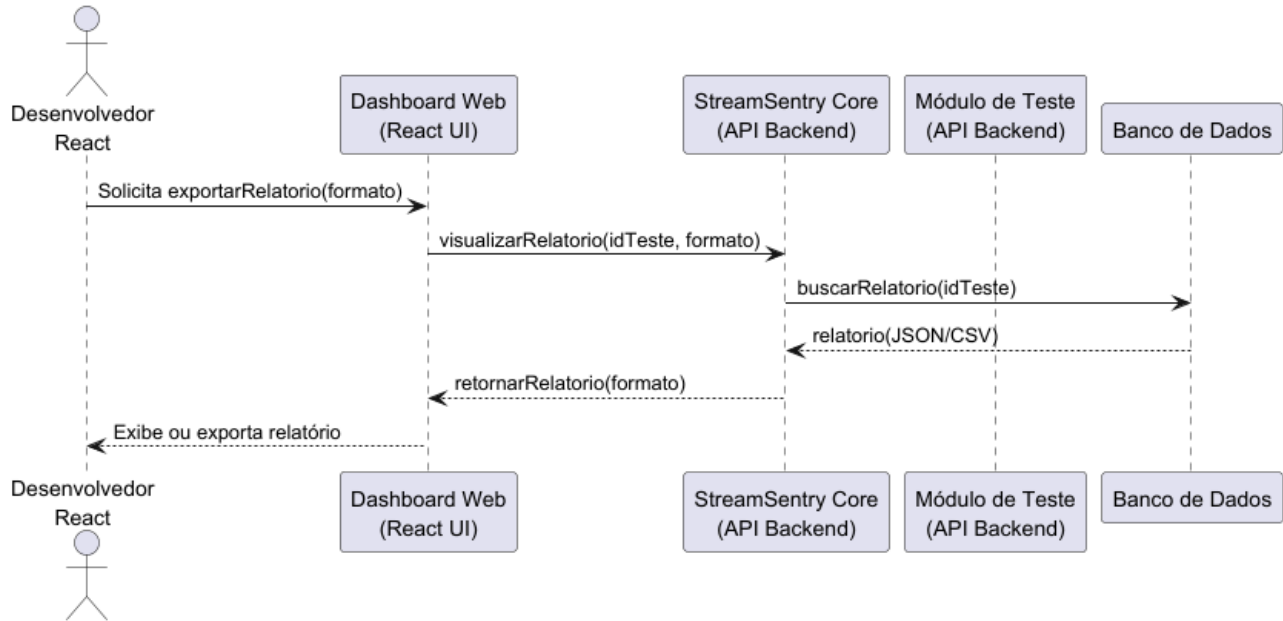


Figura 8: Diagrama de Sequência para Exportação de Relatório

Contrato	Configurar Teste
Operação	configurarTeste(quantidadeUsuarios: int, api: string, duracao: int)
Referências cruzadas	US01, US05, US06
Pré-condições	- Usuário está autenticado. - API de videoconferência (ex.: WebRTC, Jitsi) está configurada e acessível.
Pós-condições	- Parâmetros do teste (número de usuários virtuais, tipo de API, duração) são salvos. - Sessão de teste é iniciada na API de videoconferência.

Tabela 1: Tabela de Configuração de Teste

Contrato	Executar Teste
Operação	executarTeste(idTeste: int)
Referências cruzadas	US02, US05
Pré-condições	- Teste está configurado com parâmetros válidos. - API de videoconferência está ativa.
Pós-condições	- Teste é executado com sucesso, e métricas (latência, falhas) são coletadas. - Relatório é gerado e armazenado para visualização.

Tabela 2: Tabela de Configuração de Teste

Contrato	Visualizar Relatório
Operação	visualizarRelatorio(idTeste: int, formato: string)
Referências cruzadas	US03
Pré-condições	- Teste foi executado, e relatório está disponível. - Formato solicitado (JSON/CSV) é suportado.
Pós-condições	- Relatório é exibido na interface web ou exportado no formato solicitado.

Tabela 3: Tabela de Execução de Teste

3. Diagramas

A seção de Diagramas usa artefatos UML para apresentar uma visão integrada da estrutura, comportamento e arquitetura do Stream Sentry, cobrindo desde a modelagem estática até os fluxos dinâmicos e a implantação. Esses modelos sustentam a implementação, promovem a separação de responsabilidades e facilitam a compreensão, manutenção e escalabilidade do sistema em todos os seus níveis.

3.1 Diagrama de Classes

O diagrama de classes do Stream Sentry foca na classe Usuario (que guarda nome e tipo), permitindo que ele crie vários Testes e ConfiguracoesAPI. A classe Teste define os parâmetros da execução e utiliza uma ConfiguracaoAPI (com endpoints e tokens) para rodar. Cada Teste gera um único Relatorio, que contém as métricas de latência e falhas (exportáveis em JSON/CSV). Por fim, a classe APIVideoconferencia é a interface para as ferramentas externas de vídeo. Tudo está ligado de forma clara, facilitando a manutenção e a cobertura das funcionalidades.

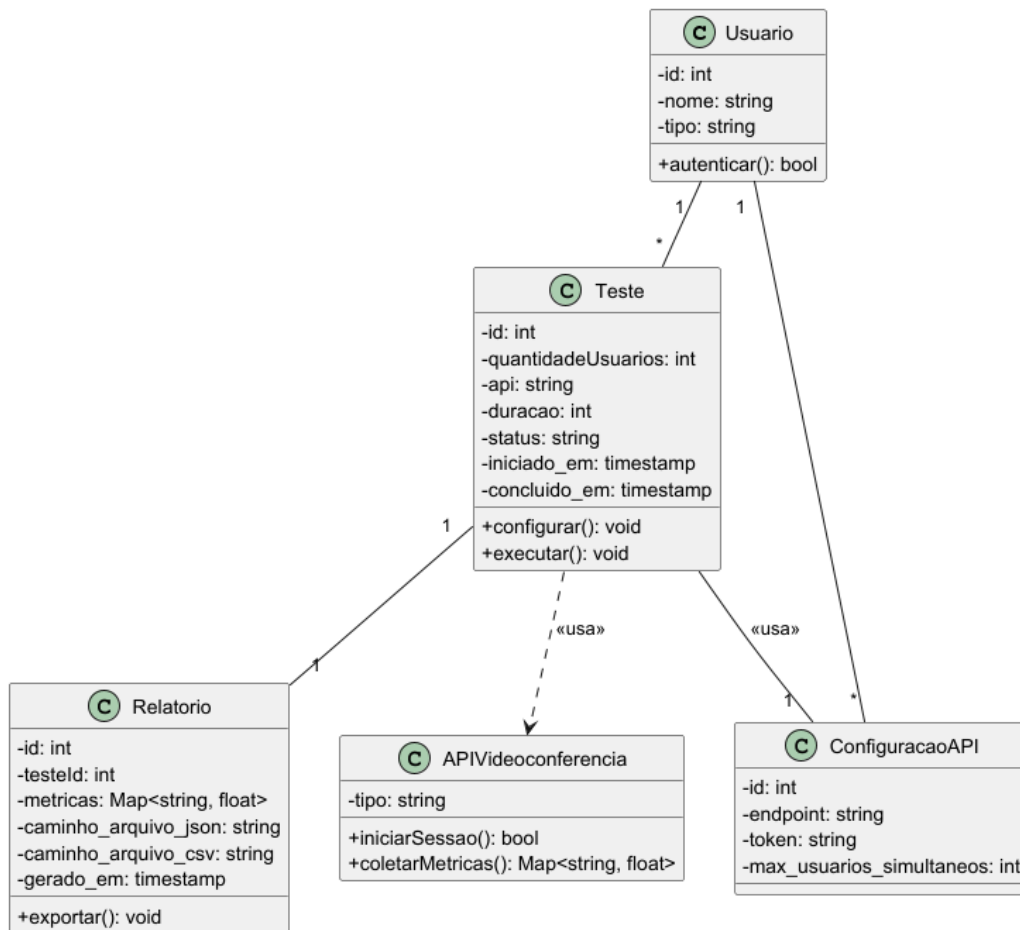


Figura 9: Diagrama de Classes do Stream Sentry

3.2 Diagramas de Sequência

O diagrama de sequência apresenta o fluxo completo do Stream Sentry em três fases: configuração, execução e relatório. O usuário interage com a interface web (React) para configurar o teste, que é salvo pelo Core (Node.js) no banco. A fase de execução é disparada por uma única chamada síncrona (POST /executar) que aguarda o término de todo o processo (passos 7 a 16). Nela, o Puppeteer cria a sala na API (Jitsi/Zoom) e entra em um loop, coletando métricas (latência, falhas) a cada 2 segundos e enviando-as individualmente ao Core para persistência. Ao final do teste, o relatório é gerado e a chamada inicial é finalmente respondida, permitindo ao usuário acessar os resultados (JSON/CSV) no fluxo de relatório subsequente.

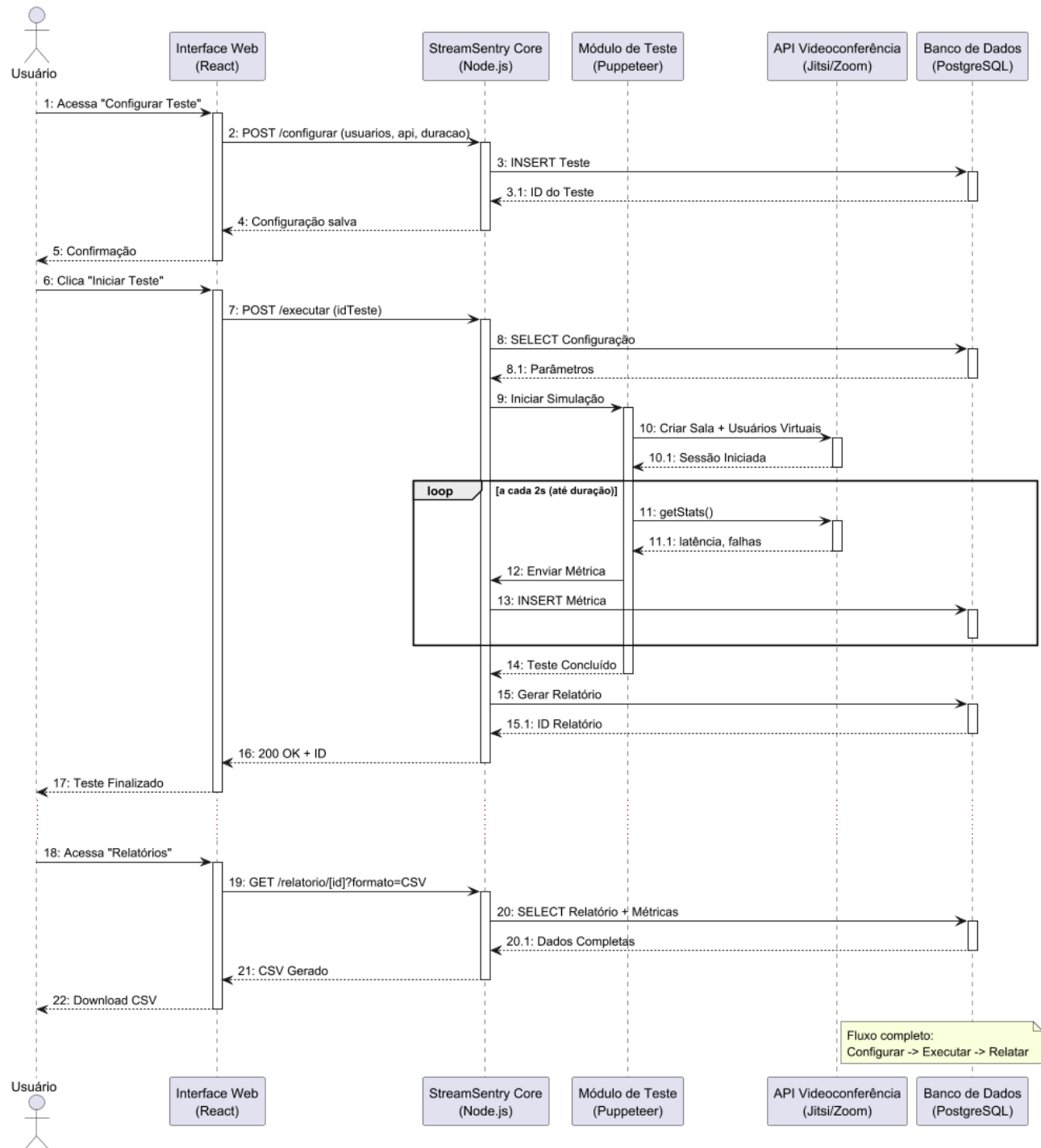


Figura 10: Diagrama de Sequência do Stream Sentry

3.3 Diagramas de Comunicação

O diagrama de comunicação mostra como os componentes do Stream Sentry colaboram durante a execução de um teste automatizado: o Engenheiro de QA inicia o processo no Core, que busca configurações no banco de dados, ativa o Módulo de Teste para criar uma sala na API Jitsi, coleta métricas em tempo real (latência e falhas) a cada 2 segundos, salva os dados e, ao final, gera um relatório que é retornado ao usuário. As mensagens são numeradas e organizadas espacialmente, destacando a interação em rede entre frontend, backend, API externa e persistência, com o loop de monitoramento indicado por uma nota lateral.

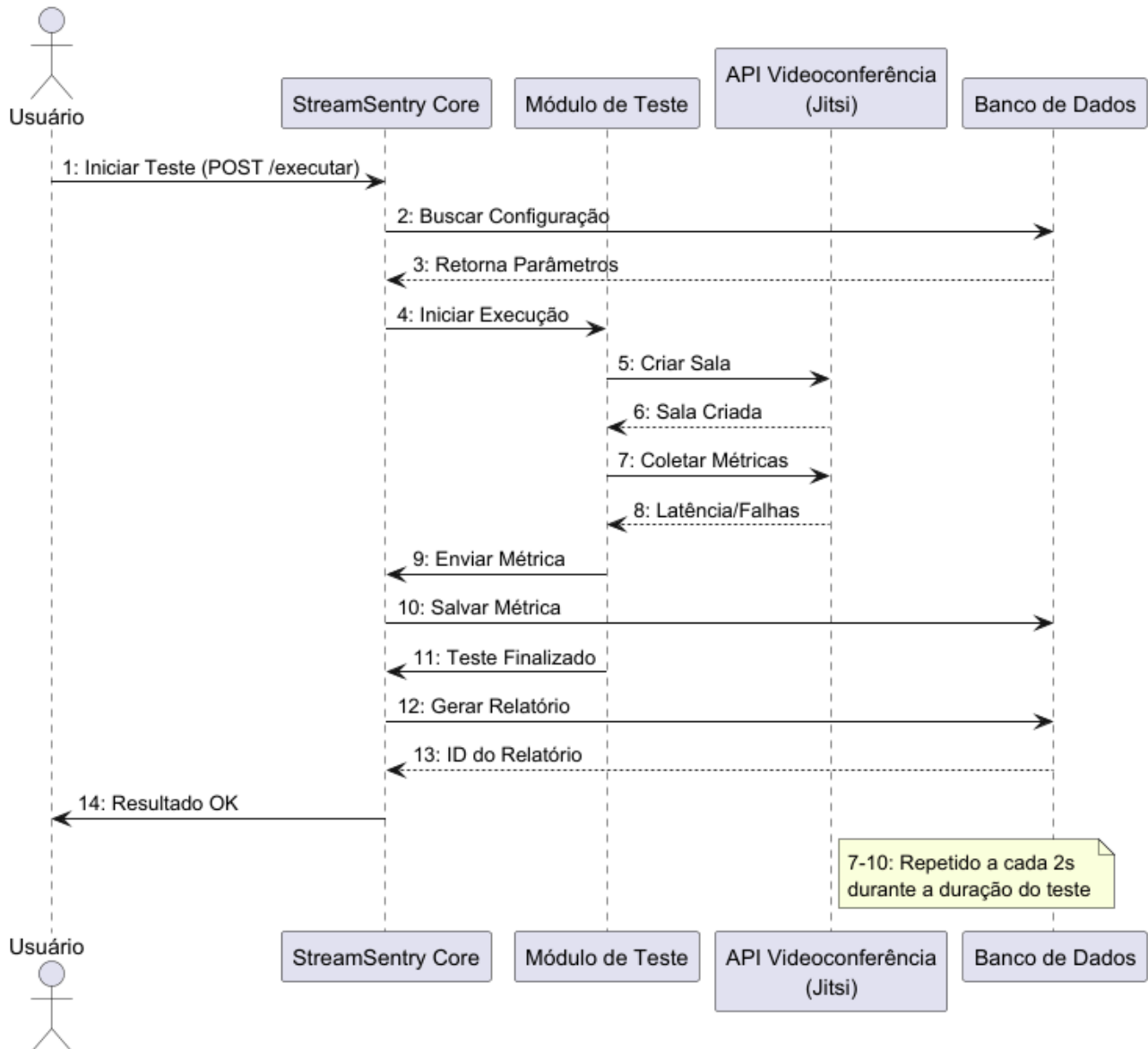


Figura 11: Diagrama de Comunicação do Stream Sentry

3.4 Arquitetura Lógica

A arquitetura lógica do Stream Sentry é representada por um diagrama de pacotes UML que organiza o sistema em três camadas principais: Interface Web (React), Stream Sentry Core (Node.js) e Integração Externa. A camada de interface contém componentes de UI como Dashboard, Configurar Teste e Relatórios, consumindo APIs REST do backend. O núcleo do sistema é dividido em controladores (TestController, ReportController), serviços de negócio (TestEngine, MetricCollector, ReportGenerator) e persistência via PostgreSQL, garantindo separação clara de responsabilidades. A integração externa abrange Jitsi, Zoom SDK e WebRTC nativo, acessados via SDKs e WebSocket. Essa estrutura promove coesão, baixa acoplamento e escalabilidade, alinhada ao Documento de Visão e aos fluxos de dados dos diagramas de sequência e comunicação.

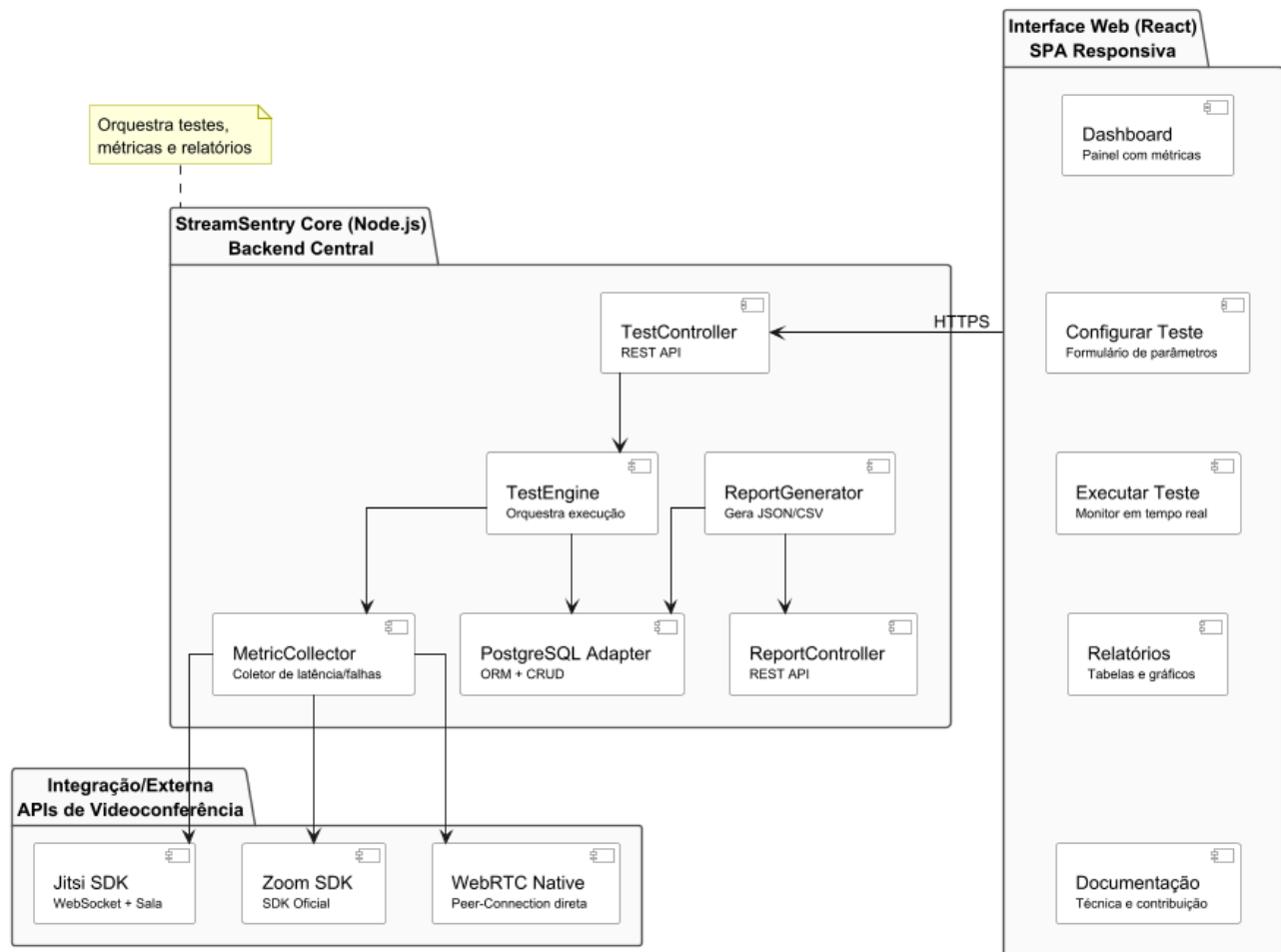


Figura 11: Diagrama de Arquitetura Lógica

3.5 Diagrama de Atividade

O Diagrama de Atividades (Figura 12) ilustra o fluxo "Configurar e Executar Teste": o usuário define os parâmetros (usuários, API, duração), que são validados e salvos no banco. A execução inicia, criando usuários virtuais e entrando em um loop de monitoramento de métricas. Ao término da duração, a sessão é finalizada, um relatório (JSON/CSV) é gerado e o usuário é redirecionado para a visualização.

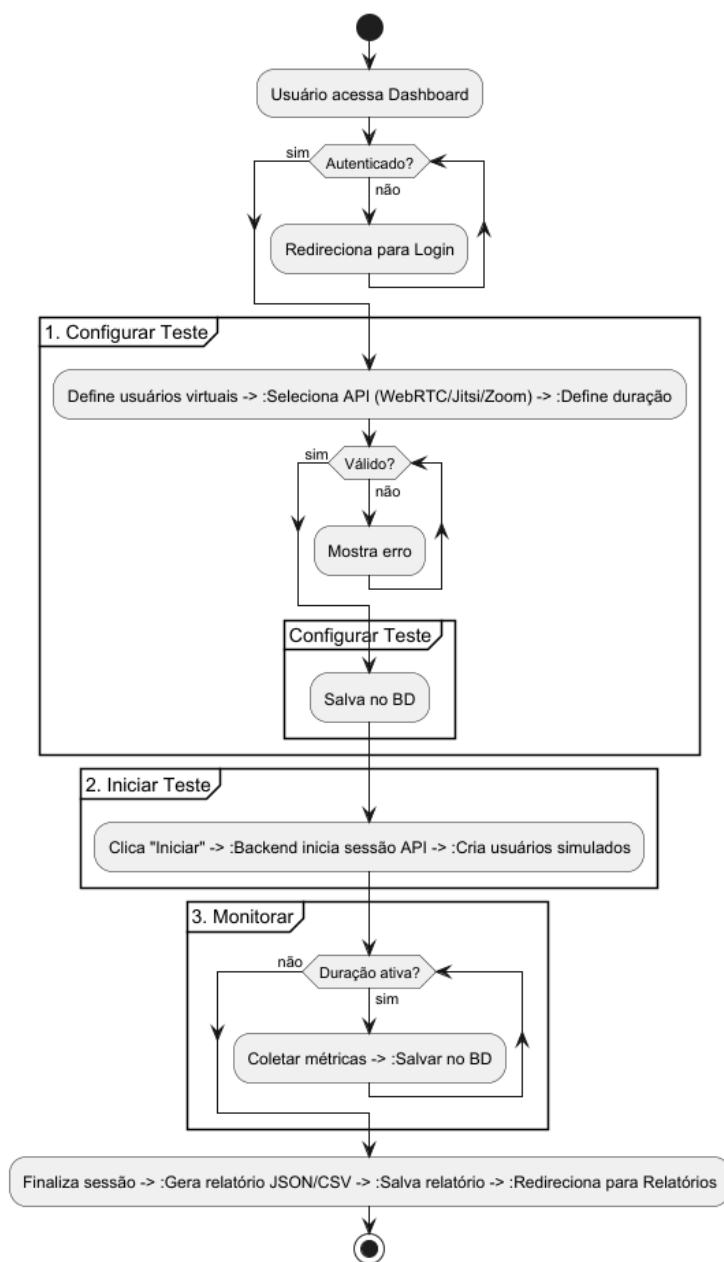


Figura 12: Diagrama de Atividade

3.6 Diagrama de Componentes e Diagrama de Implantação

O Diagrama de Componentes do Stream Sentry (Figura 13) modela os módulos reutilizáveis em três camadas: Interface Web (React) com componentes de UI (Dashboard, Configurar Teste, Executar Teste, Relatórios e Documentação), Stream Sentry Core (Node.js) com controladores REST, serviços (Test Engine, Metric Collector, Report Generator) e persistência via PostgreSQL, e Integração Externa com Jitsi, Zoom SDK e WebRTC. As interfaces REST, WebSocket e SQL garantem o desacoplamento, permitindo comunicação clara e reutilização. Essa estrutura promove manutenibilidade e escalabilidade, alinhada ao Documento de Visão e aos fluxos de dados.

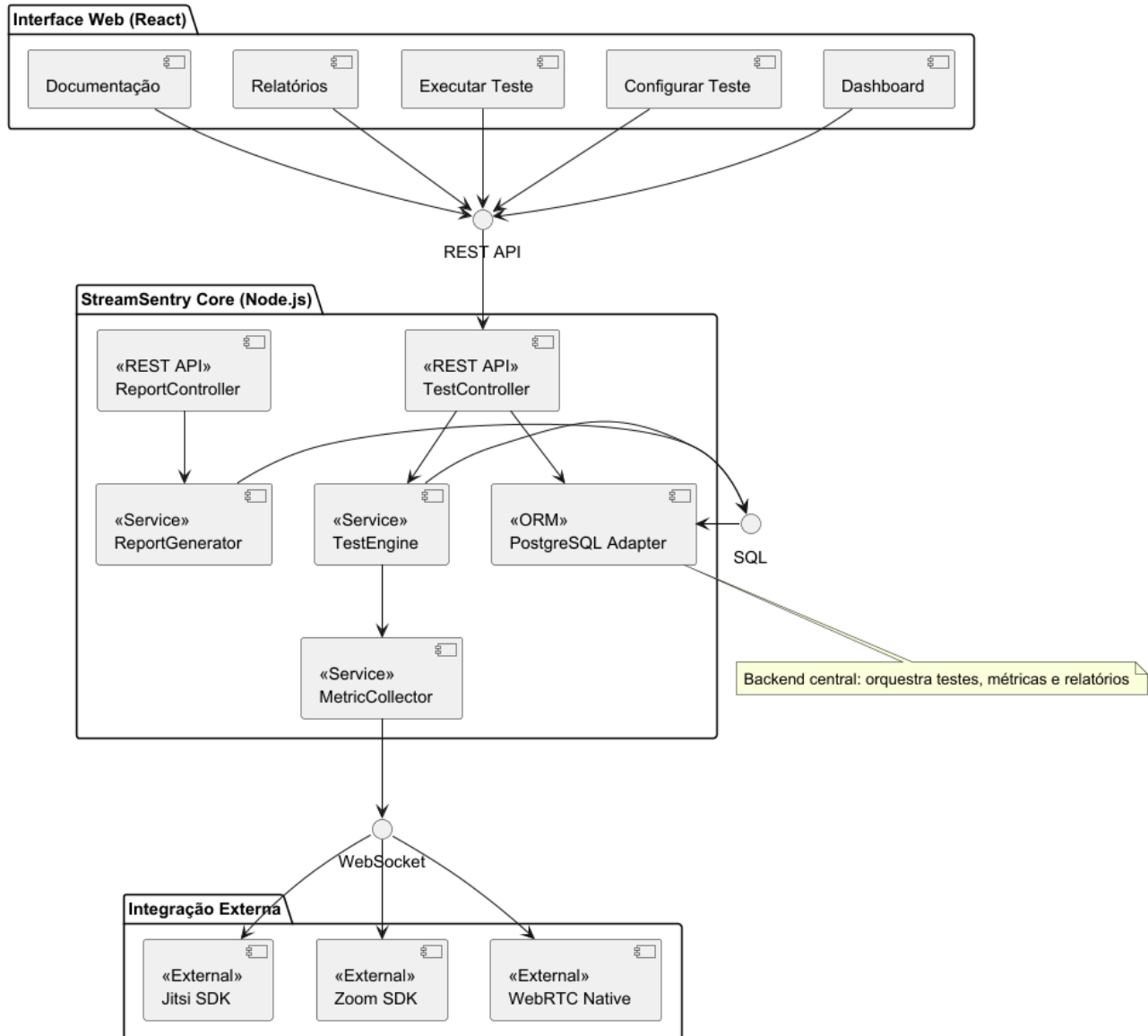


Figura 13: Diagrama de Componentes

O Diagrama de Implantação do Stream Sentry (Figura 14) define a infraestrutura física: o frontend (React SPA) é implantado em Vercel ou Netlify com build automático via GitHub, o backend (Node.js API) roda em Render ou AWS EC2 com container Docker e PM2 para alta disponibilidade, e o banco PostgreSQL é gerenciado (RDS ou Render). A comunicação ocorre via HTTPS (REST) entre frontend e backend, TCP 5432 (SQL) para persistência e WebSocket/HTTPS com APIs externas (Jitsi, Zoom, WebRTC). Essa arquitetura cloud-native garante escalabilidade, segurança e facilidade de deploy, atendendo aos requisitos de automação e integração do sistema.

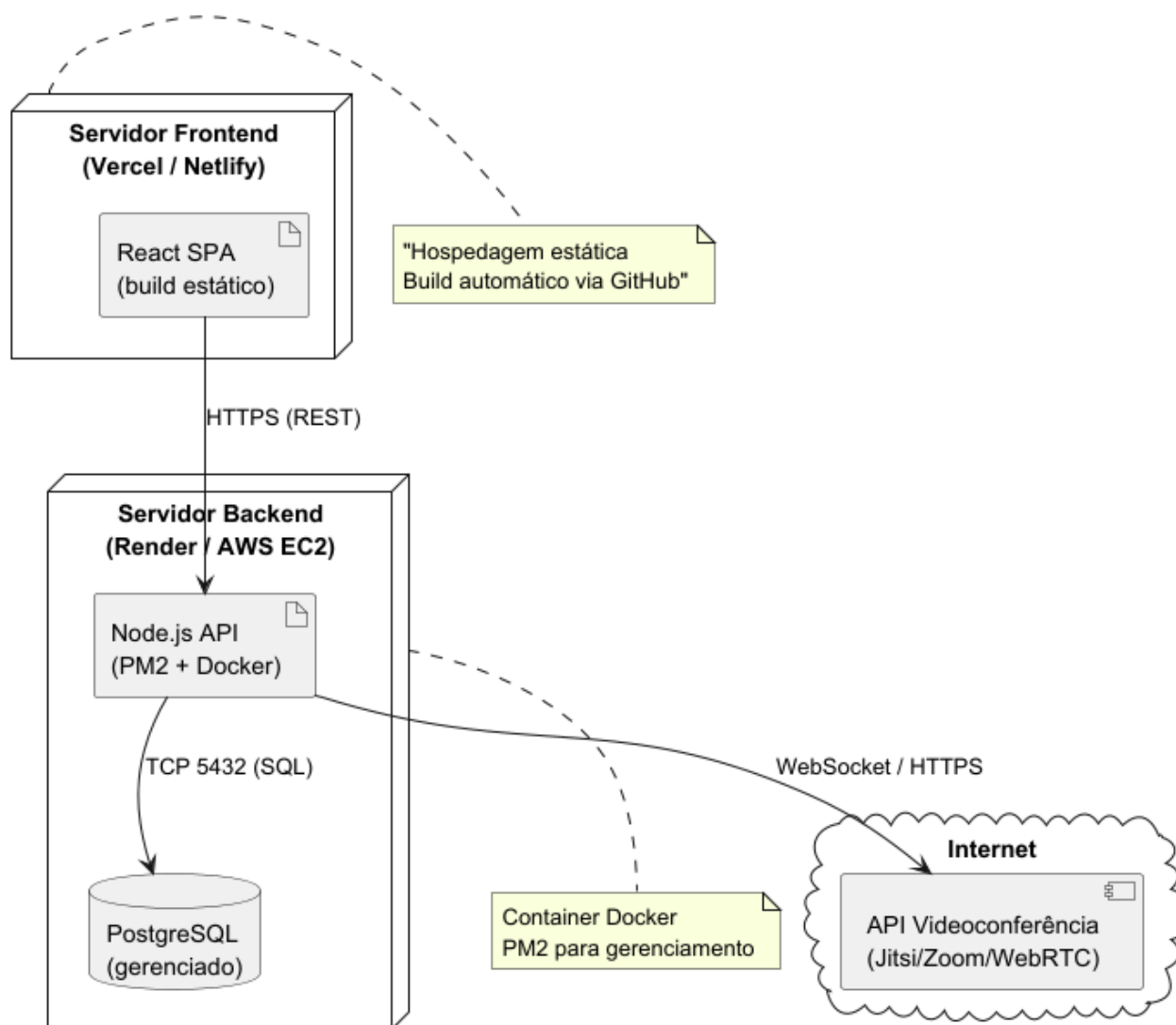


Figura 14: Diagrama de Implantação

4. Projeto de Interface com Usuário

O projeto de interface com usuário do Stream Sentry prioriza clareza técnica, usabilidade ágil e consistência visual, oferecendo uma experiência fluida para todos os atores (desenvolvedores, QA, pesquisadores e equipes ágeis). As interfaces comuns incluem um Dashboard central com visão geral de métricas, menu de navegação intuitivo e exportação de relatórios, enquanto telas específicas como Configurar Teste, Executar Teste, Relatórios, Documentação Técnica, Gerenciar Usuários, Histórico, Configurações Avançadas, Suporte e Login/Registro são projetadas com layouts responsivos, validação em tempo real, temas técnicos (modo escuro, terminais) e integração com GitHub. O design é minimalista, funcional e inspirado em ferramentas modernas (React Docs), garantindo acessibilidade, rastreabilidade e foco na automação de testes multimídia, com figuras ilustrativas que reforçam a coerência entre funcionalidade e experiência do usuário.

4.1 Interfaces Comuns a Todos os Atores

A tela inicial do sistema Stream Sentry oferece uma visão geral das operações e métricas de teste, com um menu superior contendo as opções Configurar Teste, Executar Teste, Relatórios e Documentação, um painel central exibindo gráficos e indicadores de desempenho como latência média, taxa de falhas e cobertura de testes, uma barra lateral com atalhos para configurações avançadas e acesso ao código-fonte, além de um botão de exportação para gerar relatórios em JSON ou CSV, tudo em um layout minimalista, responsivo e focado em clareza e usabilidade técnica.

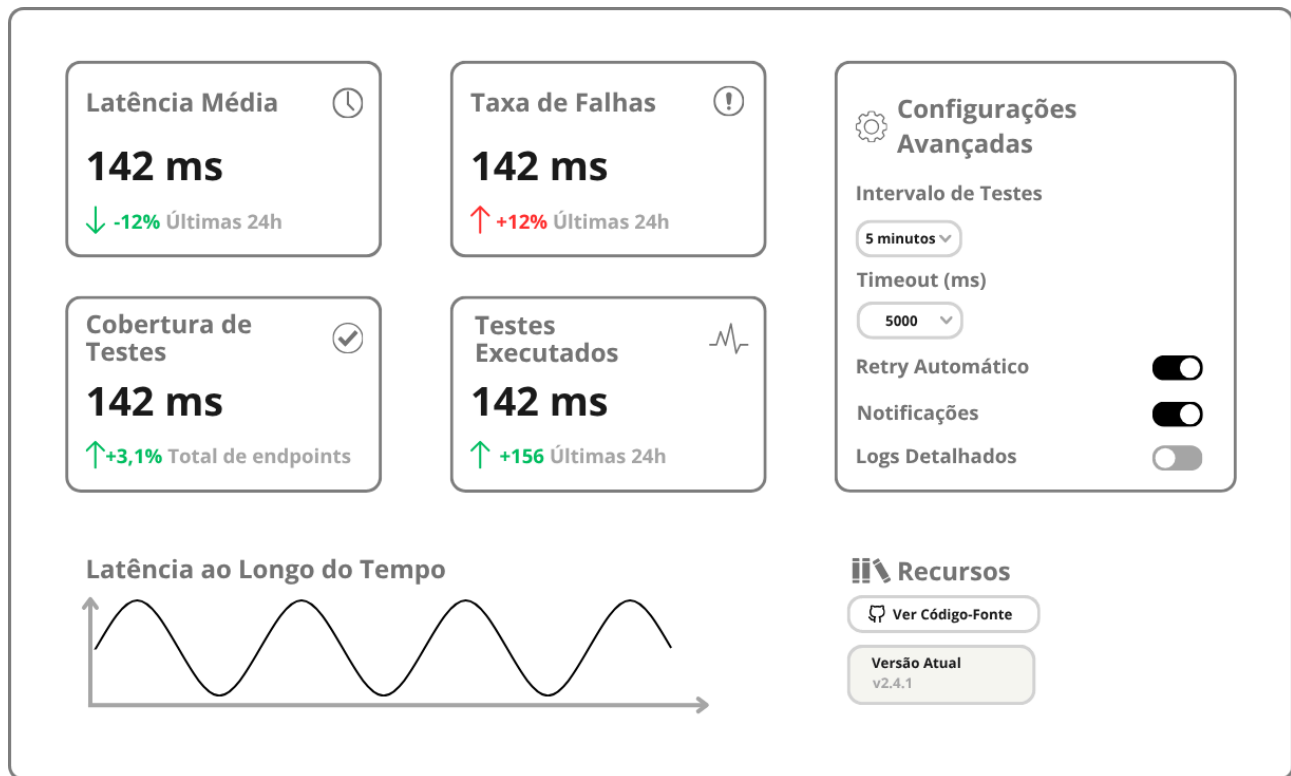


Figura 15: Tela Principal – Dashboard

Esta tela permite ao usuário definir os parâmetros de um novo teste automatizado, com campos para selecionar a quantidade de usuários virtuais (via controle deslizante), a API de videoconferência (WebRTC, Jitsi ou Zoom SDK) e a duração do teste (em minutos), exibindo indicadores de status como API conectada e usuários simulados prontos, além de botões “Salvar Configuração” e “Iniciar Teste” para execução direta, tudo em um design simples, com validação rápida de entradas e otimizado para fluxos ágeis.

A interface 'Configurar Teste' apresenta os seguintes elementos:

- Status da API:** Um ícone de câmera verde indica o status 'Conectada' com o texto 'Conexão estabelecida'.
- Usuários Simulados:** Um ícone de três pessoas azuis indica '5/5' instâncias ativas.
- Parâmetros do Teste:** Uma seção contendo:
 - Quantidade de Usuários Virtuais:** Um controle deslizante azul com o valor '35' exibido em um botão à direita.
 - API de Videoconferência:** Um menu suspenso com o texto 'Selecione a API para teste' e uma seta para baixo.
 - Duração do Teste:** Um campo de entrada com um ícone de relógio, o valor '10' e a unidade 'minutos'.

Figura 16: Tela “Configurar Teste”

Esta tela é responsável por monitorar a execução de testes em tempo real, exibindo um painel de logs com informações de desempenho como latência, bitrate e falhas, além de um gráfico dinâmico que mostra variações ao longo do tempo, uma barra lateral de progresso indicando o status da execução e botões para pausar, finalizar ou gerar relatório, tudo em um visual com tema escuro inspirado em terminais técnicos, transmitindo precisão e controle.

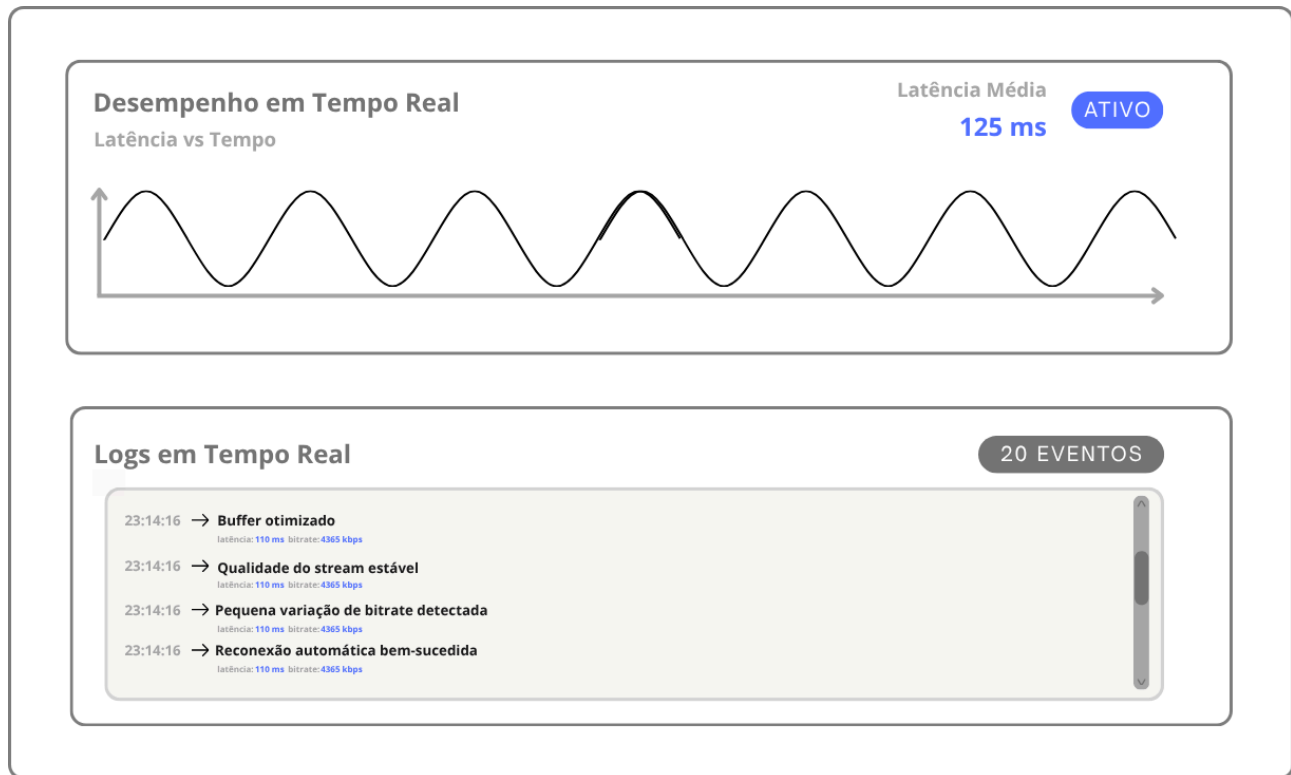


Figura 17: Tela “Executar Teste”

Esta tela centraliza o acesso aos resultados dos testes realizados, apresentando uma tabela com histórico de execuções incluindo API utilizada, número de usuários e status (Sucesso/Falha), exibindo ao selecionar um teste gráficos detalhados com métricas de latência, falhas e qualidade de vídeo/áudio, com opções para filtrar resultados por data, tipo de API ou status e exportar relatórios em JSON ou CSV, tudo em um layout de painel analítico com clareza visual e foco na interpretação rápida de dados.

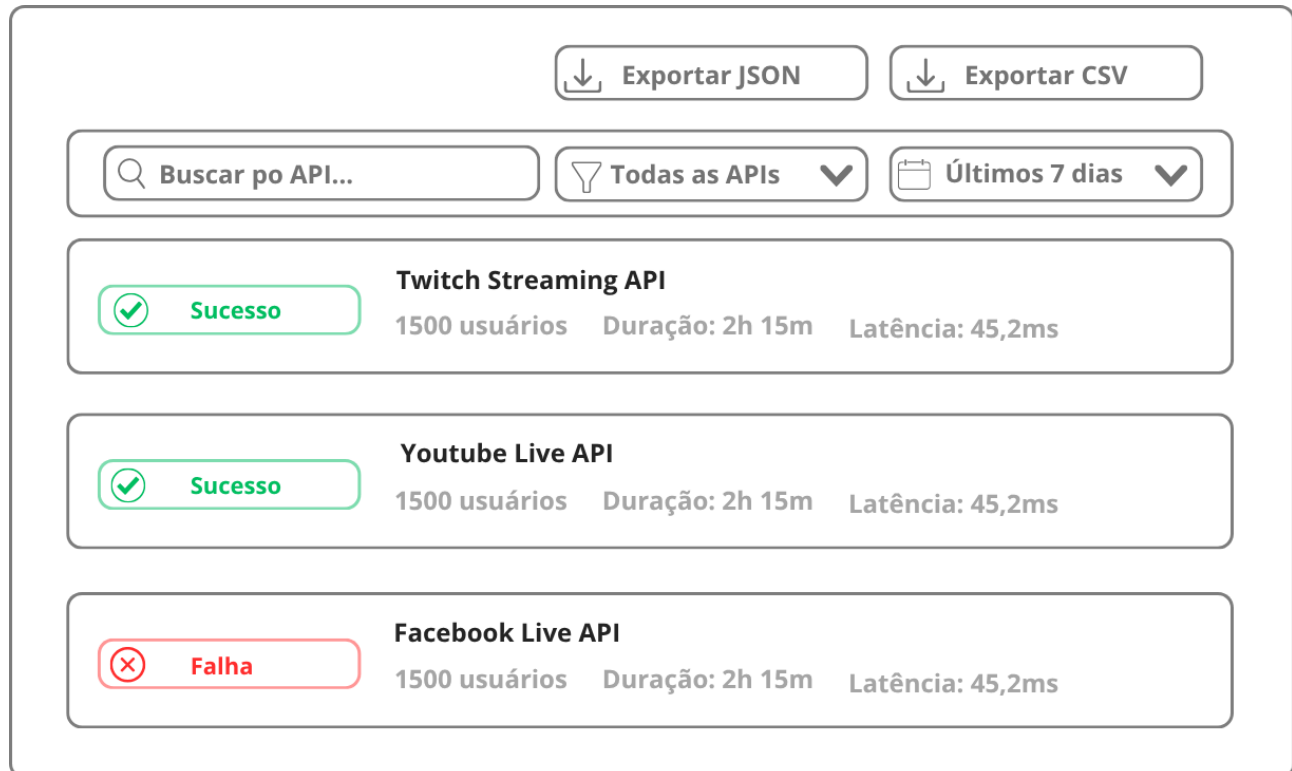


Figura 18: Tela “Relatórios”

Esta tela exibe uma linha do tempo vertical com todos os testes realizados, API usada, duração e status visual (ícones de sucesso/falha), com cada item contendo um link direto para o relatório detalhado, em um estilo inspirado em pipelines de integração contínua como o GitHub Actions, transmitindo automação e rastreabilidade.

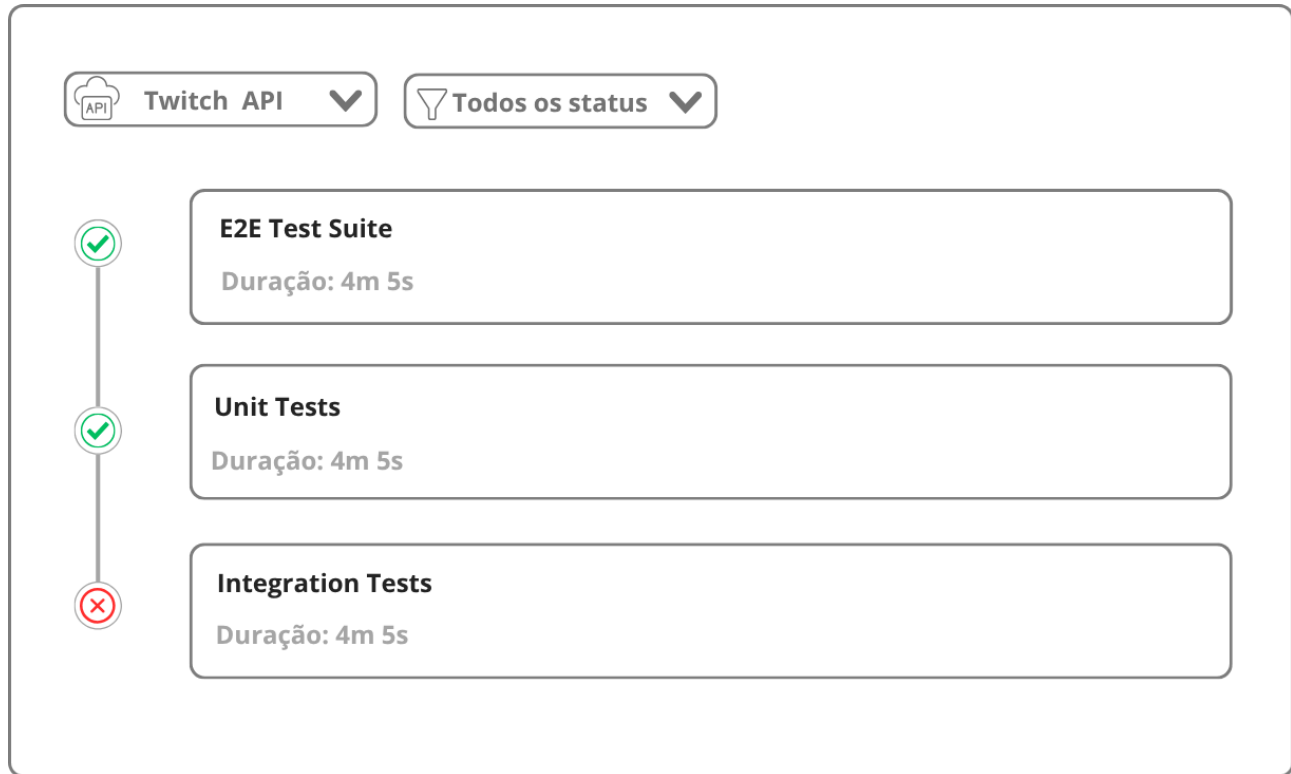


Figura 21: Tela “Histórico de Testes”

Esta tela permite ajustar parâmetros técnicos do sistema, incluindo campos para definir endpoints de APIs, tokens de autenticação e limites de usuários virtuais simultâneos, com um botão “Salvar” para aplicar as alterações, organizada em blocos separados por categoria para priorizar configuração rápida e segura.

**Endpoint da API**
Configure o endpoint da API de videoconferência

URL do Endpoint



**Token de Autenticação**
Configure as credenciais de acesso à API

Token de acesso

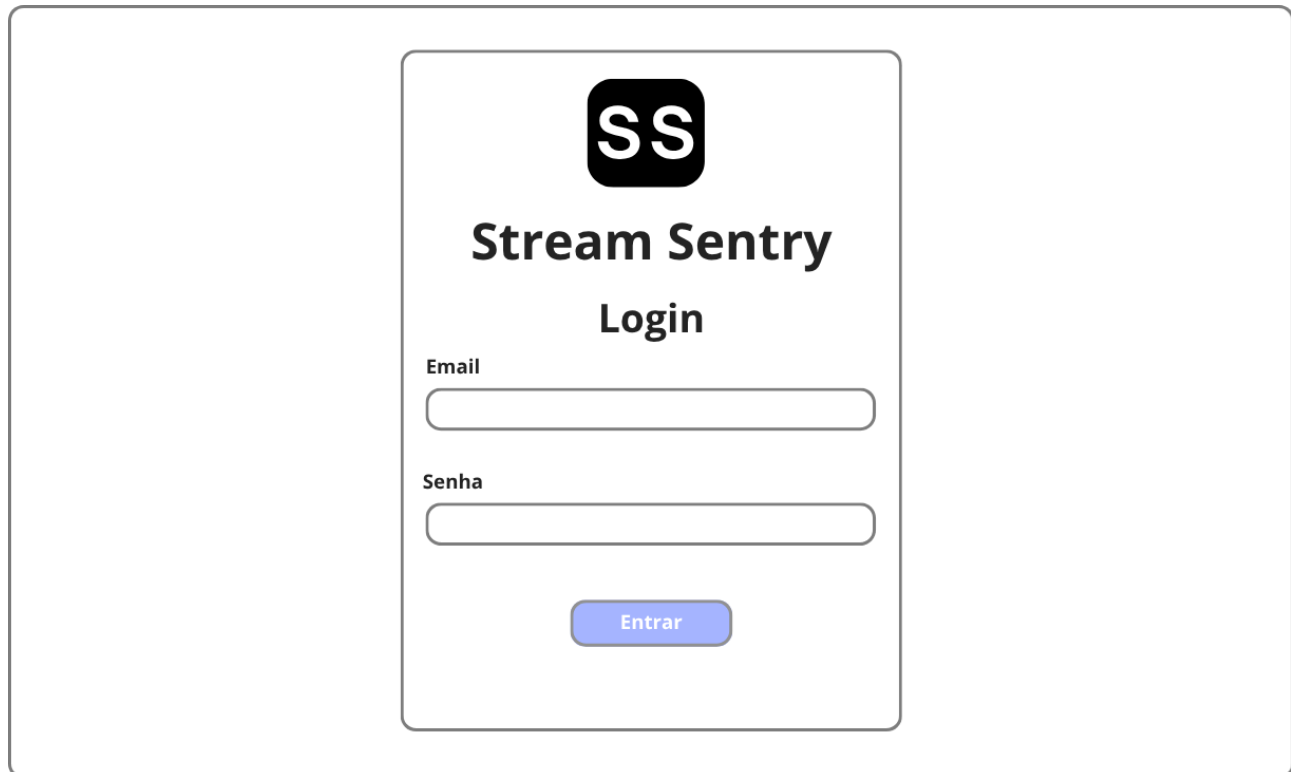
**Usuários Virtuais**
Defina o número máximo de usuários virtuais simultâneos

Limite Máximo	Atual	Em Uso	Disponível
100	100	23	77

 Salvar

Figura 22: Tela “Configurações Avançadas”

Esta tela de login e registro apresenta um design minimalista com fundo neutro, logo centralizada e campos de autenticação (email e senha), e confirmação de senha, com botões “Entrar” em estilo uniforme e feedback visual claro, visando facilitar o acesso rápido ao sistema e manter coerência visual com o restante da aplicação.



The image shows a mockup of a login screen for a system called "Stream Sentry". The screen is centered within a light gray rectangular frame. At the top center is a black square logo with the white letters "SS". Below the logo, the text "Stream Sentry" is displayed in a bold, black, sans-serif font. Underneath that, the word "Login" is centered in a smaller, bold, black, sans-serif font. Below the title, there are two input fields. The first is labeled "Email" in a small, gray, sans-serif font, and the second is labeled "Senha" (Password) in the same font. Both labels are positioned to the left of their respective input fields. The input fields are white with a thin gray border and rounded corners. Below the "Senha" field, there is a blue button with the word "Entrar" (Enter) in white, sans-serif font. The button has rounded corners and a slight gradient.

Figura 24: Tela “Login e Registro”

5. Modelos de Projeto

A seção de Modelos de Projeto apresenta os artefatos detalhados que sustentam a arquitetura e a implementação do Stream Sentry, garantindo consistência entre o Documento de Visão, os diagramas UML e a base de código. O modelo de dados é relacional e formalizado pelo Diagrama de Entidade e Relacionamento (DER) (Figura 25), composto por quatro entidades principais — Usuario, Teste, Relatorio e ConfiguracaoAPI — com relacionamentos 1:N e 1:1, chaves primárias UUID e tipos ENUM para integridade e eficiência. Esse modelo suporta todas as operações do sistema (configurar, executar, relatar) e será implementado em PostgreSQL com ORM (TypeORM/Prisma), sendo consistente com o Diagrama de Classes (Seção 3.1). O glossário de atributos (Tabela 1) lista todos os campos reconhecidos nas entradas (configuração de testes) e saídas (relatórios), com formato, restrições e descrição clara, facilitando a validação de dados e a comunicação entre frontend, backend e banco. Juntos, esses modelos promovem normalização, manutenibilidade e escalabilidade, atendendo aos requisitos funcionais e não funcionais do sistema.

5.1 Modelo de Dados

O modelo de dados do Stream Sentry é relacional e representado pelo Diagrama de Entidade e Relacionamento (DER) (Figura 16), composto por quatro entidades principais: Usuario (com atributos de autenticação e função), Teste (configuração e execução com status gerenciado por flags), Relatório (resultados e arquivos exportados) e Configuração API (credenciais de integração). Os relacionamentos são 1:N entre Usuario e Teste, 1:1 entre Teste e Relatório, e 1:N entre Usuario e Configuração API. O uso de chaves primárias UUID, chaves estrangeiras e tipos ENUM garante integridade referencial, normalização e eficiência em consultas. O modelo suporta todas as operações do sistema (configurar, executar, relatar), é consistente com o Diagrama de Classes (Seção 3.1) e será implementado em PostgreSQL com ORM (TypeORM/Prisma).

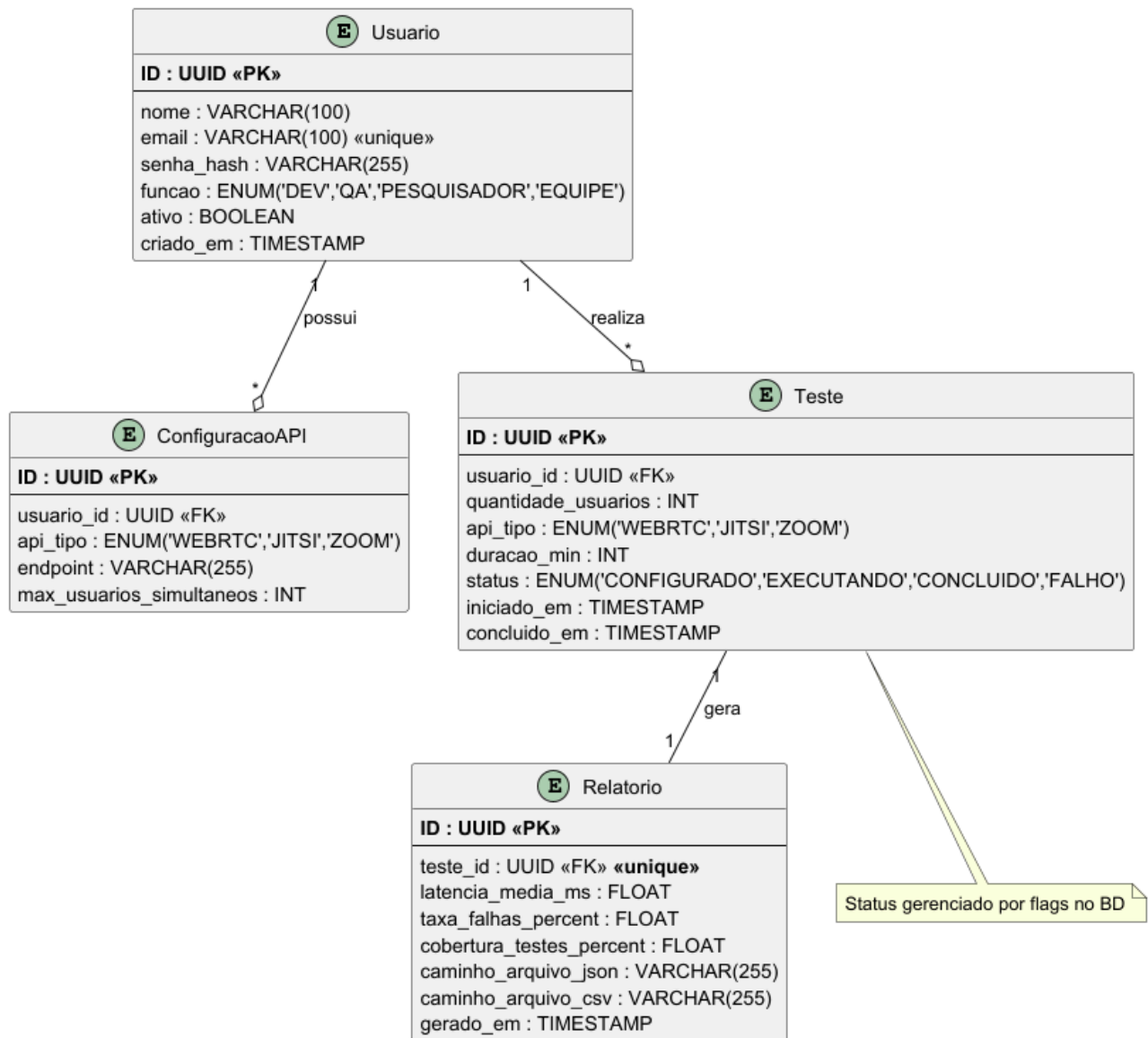


Figura 25: Diagrama de Entidade e Relacionamento

5.2 Glossário

Atributo	Formato	Descrição
quantidade_usuarios	INT (1–500)	Número de usuários virtuais simulados no teste.
api_tipo	ENUM('WEBRTC','JITSI','ZOOM')	Tipo de API de videoconferência utilizada no teste.
duracao_min	INT (1–1440)	Duração do teste em minutos.
status	ENUM('CONFIGURADO','EXECUTANDO','CONCLUIDO','FALHO')	Estado atual do teste no sistema.
latencia_media_ms	FLOAT	Média de latência de rede durante o teste (em milissegundos).
taxa_falhas_percent	FLOAT (0.0–100.0)	Percentual de falhas de conexão ou pacotes perdidos.
cobertura_testes_percent	FLOAT (0.0–100.0)	Percentual de funcionalidades da API cobertas pelo teste.
caminho_arquivo_json	VARCHAR(255)	Caminho ou URL do relatório gerado em formato JSON.
caminho_arquivo_csv	VARCHAR(255)	Caminho ou URL do relatório gerado em formato CSV.
endpoint	VARCHAR(255)	URL base da API de videoconferência.

Atributo	Formato	Descrição
token	TEXT	Token de autenticação para APIs que exigem (Zoom, Jitsi custom).
max_usuarios_simultaneos	INT	Limite máximo de usuários virtuais por execução.
iniciado_em	TIMESTAMP	Data e hora do início da execução do teste.
concluido_em	TIMESTAMP	Data e hora do término da execução do teste.
gerado_em	TIMESTAMP	Data e hora da geração do relatório.

Tabela 4: Glossário

6. Cronograma

O cronograma do TCC II será executado em um modelo Ágil, estruturado em dez Sprints de duas semanas cada, cobrindo o desenvolvimento completo do software e a documentação técnica até o final de junho de 2026. Este planejamento prioriza o objetivo central do TCC: as Sprints iniciais (S1 a S5) focam no Core, Métricas e Integrações com todas as APIs de vídeo. As Sprints seguintes (S6 a S8) dedicam-se à Interface Web e aos Testes Integrados (E2E). Por fim, as Sprints finais (S9 e S10) são reservadas à Coleta de Métricas Definitivas e à Redação dos Capítulos de Resultados da monografia, garantindo que o produto técnico e os dados para o TCC estejam consolidados no prazo.

Mês	Período (Quinzenas)	Sprint	Foco Principal	Atividades Detalhadas
Fev/26	1ª Quinzenal (1-15)	S1	Setup e Core Base	Configuração do ambiente (Node.js/PostgreSQL). Implementação dos modelos de dados e do TestController.
	2ª Quinzenal (16-29)	S2	Motor de Testes	Implementação do TestEngine e MetricCollector base. Primeira execução funcional do teste com WebRTC Native.

Mar/26	1ª Quinzenal (1-15)	S3	Coleta de Métricas	Finalização do MetricCollector. Implementação das rotinas de coleta de latência e taxa de falhas e persistência no BD.
	2ª Quinzenal (16-31)	S4	Integração de APIs	Implementação da conexão com Zoom SDK e Jitsi SDK . Validação da coleta de métricas nas APIs integradas.
Abr/26	1ª Quinzenal (1-15)	S5	Relatórios RAW e Docs	Implementação do ReportGenerator. Geração do Relatório em formato RAW (JSON/CSV). Finalização do Capítulo 3 (Metodologia).
	2ª Quinzenal (16-30)	S6	Interface Essencial	Desenvolvimento das telas Configurar Teste e Executar Teste (monitoramento em tempo real) no React.
Mai/26	1ª Quinzenal (1-15)	S7	Visualização de Dados	Desenvolvimento das telas Relatórios e Dashboard no React, incluindo visualização de gráficos e exportação de dados.
	2ª Quinzenal (16-31)	S8	Testes Integrados (E2E)	Execução da primeira rodada de Testes Integrados (E2E) completos em todos os fluxos e APIs. Correção de <i>bugs</i> críticos.
	1ª Quinzenal (1-15)		Qualidade e Docs Técnica	Segunda rodada de Testes Integrados . Elaboração da Documentação Técnica

Jun/26		S9		(Arquitetura, APIs) e do Capítulo 4 (Resultados/Análise V1).
	2ª Quinzenal (16-30)	S10	Fechamento e Resultados Finais	Fechamento da Versão 1.2. Coleta de Métricas Definitivas para o TCC. Finalização do Capítulo 5 (Conclusão e Trabalhos Futuros).

Tabela 5: Tabela de Cronograma